



Jawwal Pay System Service Bus POS Developer Guide

Version 1.0



1 Contents

1	Introduction	3
1.1	Authentication Tokens and Security.....	3
1.2	API Requests Format.....	3
1.3	URL Endpoints.....	3
2	API Requests	3
2.1	Login.....	3
2.2	v1/get_balance	4
2.3	v1/business/send_otp	4
2.4	v1/business/MFP	5
2.5	v1/business/generate_qr	5
2.6	v1/business/search_trans.....	6
3	Calculate secure hash:	7
	3- Calculate the hash of the concatenated string using SHA512 and your HMAC secret provided to you by	7
4	Error Codes	8
5	History of changes	10



1 Introduction

This document describes the national company for electronic payments service bus interface,

1.1 Authentication Tokens and Security

All communications with the Jawwal Pay service bus are encrypted with an external layer of encryption "SSL". Once you are enrolled in the System Program, you will be provided with a unique username, password, apiKey for both test and production environments. The production environment login info will be provided upon successful completion of your integration on test environment.

Before requesting any API, you are required to login to the system using login info provided above, then you'll be provided with a token, this token has an expiry time (configurable), and this token should be sent on each request in request header (X-Auth-Token).

Also a secureHash should be developed per each request and passed to each request except the login API in the body as explained in section 3.

1.2 API Requests Format

Requests should be made using the **POST method** unless otherwise stated for the given service.

Response data is provided in JSON format. All data is encoded using the UTF-8 character set. Requesting an unknown service will return a '404 Not Found' response. For known API requests the bus returns a detailed error code and error description to help in handling API errors on the Client's System. Please ensure to capture the error message in the response.

1.3 URL Endpoints

The URL endpoint is created by combining the base URL of the environment in which you want to execute your request; Test vs. Production, the version of the API, the appropriate login info for that environment, and the function you want to execute along with the posted parameters as shown in the example below.

For example, if you want to execute jawwal topup request in the Test environment, you would specify the following URL when executing an HTTPs POST:

https://apitest.jawwalpay.ps/v1/business/send_OTP

Posted Parameters:

- 1) Request parameters.
- 2) Authentication Token in X-Auth-Token header.
- 3) SecureHash in the request body.

2 API Requests

Every API Request will require the user token to be passed in the request header.

The set of Parameters, which varies from one Request to another, need to be converted to JSON.

The results will be returned in JSON format.

2.1 Login

<https://apitest.jawwalpay.ps/login>

This function is used to login the user into Jawwal Pay system, user will be granted a token, this token will be used as a parameter for all other functions, if the token is expired, user has to login again and acquire a new token.

Basic authorization header is used, the credentials are constructed like this:

The username and the password are combined with a colon (user:password).

The resulting string is base64 encoded (dXNlcjpwYXNzd29yZA==).



Request Example:

Authorization: Basic dXNlcjpwYXNzd29yZA==

Returned Parameters:

#	Parameter	Type	Required	Description
1	errorCd	String	M	Will return "00" if user logged in successfully and error code if not logged in.
2	desc	String	M	Will return "Success" if successful and "Error message" if not logged in successful
3	extraData -> key: access_token	String	O	Token generated by the Jawwal Pay system, used to authenticate and authorize the users in the next API calls.
4	ref	String	M	Response message reference
5	statusCode	String	M	1 if success and 3 for failure

Result Example:

If login successful:

```
{ "errorCd": "00", "desc": "Success", "extraData": { "key": "access_token", "value": "e7uhdjsdjh3ejdndndjdjdhdjd...." }, "ref": "123456789", "statusCode": "1" }
```

If login fails:

```
{ "errorCd": "45", "desc": "Not authorized", "ref": "123456789", "statusCode": "3" }
```

2.2 v1/get_balance

This function is used to list all user accounts and their balances

Required Parameters:

Parameter	Type	Required	Description
msgId	String	M	Unique message id
lang	String	O	AR for Arabic and EN for English : if not present, Arabic is selected
secureHash	String	M	See appendix 3

Request Example:

```
{ "msgId": "001", "lang": "EN", "secureHash": "23dfgj37dj38dsj8sd88d...." }
```

Response Example:

```
{ "desc": "Success", "errorCd": "00", "extraData": { "key": "info", "value": { "fullName": "ooo", "avatar": "", "adminService": "SUPERSTAR", "mobileNumber": "00970599001643", "bankName": "TNB", "bankAccount": "012395173", "isBankAccountVerified": true, "isHQ": true, "accounts": [ { "accountNumber": "1971", "accountType": "WALLET", "accountCategory": "MERCHANT", "balance": 835.20, "registrationId": "105003" }, { "accountNumber": "1963", "accountType": "WALLET", "accountCategory": "AGENT", "balance": 51354.84, "registrationId": "105001" }, { "accountNumber": "1964", "accountType": "COMMISSIONS", "accountCategory": "COMMISSION", "balance": 918.86, "registrationId": "105002" } ], "isBiller": false, "isSuperAgent": true } }, "ref": "1633629255671", "statusCode": "1" }
```

2.3 v1/business/send_otp

This function is used to send otp to customer to be used later for request money

Required Parameters:

Parameter	Type	Required	Description
msgId	String	M	Unique message id
receiver	String	M	customer mobile number 05XXXXXXX
amount	String	M	Amount to be transferred in ILS
lang	String	O	AR for Arabic and EN for English : if not present, Arabic is selected
secureHash	String	M	See appendix 3



Request Example:

```
{ "msgId": "002", "receiver": "00970598251590", "amount": "500", "lang": "EN", "secureHash": "23dfgj37dj38dsj8sd88d...." }
```

Response Example:

```
{ "desc": "Success", "errorCd": "00", "extraData": [], "ref": "2921218715102", "statusCode": "1" }
```

2.4 v1/business/MFP

This function is used to request money from customer as pay to merchant

Required Parameters:

Parameter	Type	Required	Description
msgId	String	M	Unique message id
receiver	String	M	customer mobile number 05XXXXXXXX
amount	String	M	Amount to be transferred in ILS
otp	String	M	One time password sent to customer mobile through send_otp, hashed in SHA-256 and encoded in base64
additionalBillNo	String	O	Bill number for which payment is requested (for merchant use)
additionalReferenceLabel	String	O	Transaction reference number at merchant system (for merchant use)
additionalMobileNumber	String	O	Mobile number of POS or cashier (for merchant use)
note	String	O	Notes filled by merchant (for merchant use)
promoCode	String	O	Promo code for this payment (for merchant use)
lang	String	O	AR for Arabic and EN for English : if not present, Arabic is selected
secureHash	String	M	See appendix 3

Request Example:

```
{ "msgId": "003", "receiver": "00970598251590", "amount": "500", "otp": "i/ABNulm39IL8cn0AcnPBTDRRDm5xpHJXWoo6kRfVnA\u003d", "lang": "EN", "secureHash": "23dfgj37dj38dsj8sd88d...." }
```

Response Example:

```
{ "desc": "Success", "errorCd": "00", "extraData": [], "ref": "2921218715102", "statusCode": "1" }
```

2.5 v1/business/generate_qr

This function is used to generate static or dynamic QR

Required Parameters:

Parameter	Type	Required	Description
msgId	String	M	Unique message id
initiationMethod	String	M	11: static QR, 12 dynamic QR
transactionAmount	String	C	Mandatory in case QR is dynamic, represents transaction amount in ILS currency
additionalStoreLabel	String	O	
additionalTerminalLabel	String	O	
additionalBillNo	String	O	Bill number for which payment is requested (for merchant use)
additionalReferenceLabel	String	O	Transaction reference number at merchant system (for merchant use)
additionalMobileNumber	String	O	Mobile number of POS or cashier (for merchant use)
note	String	O	Notes filled by merchant (for merchant use)
promoCode	String	O	Promo code for this payment (for merchant use)
lang	String	O	AR for Arabic and EN for English : if not present, Arabic is selected
secureHash	String	M	See appendix 3

Request Example:



```
{ "msgId": "004", "initiationMethod": "12", "transactionAmount": "100", "note": "test notes", "additionalReferenceLabel": "95se334", "promoCode": "test code", "additionalStoreLabel": "anything", "additionalMobileNumber": "0599887744", "additionalBillNo": "djneej", "additionalTerminalLabel": "deviceee", "lang": "EN", "secureHash": "23dfgj37dj38dsj8sd88d...." }
```

Response Example:

```
{ "desc": "Success", "errorCd": "00", "extraData": { "key": "qr", "value": "00020101021226220008JPAYPS22010610500152045599530337654031005802PS5906UBS-HQ6021Ramallah and Al-Bireh62920106djneej021005998877440309ana 3ayef0409test code050795se3340601C0708deviceee0810test notes64360002AR0106ubs-hq0216 رام الله 80050001263043 وفيرة F8C" }, "ref": "2921246439222", "statusCode": "1" }
```

2.6 v1/business/search_trans

This function is used to search for transactions

Required Parameters:

Parameter	Type	Required	Description
msgId	String	M	Unique message id
senderAccountCategory	String	O	Account category (AGENT, MERCHANT, COMMISSION)
Sender	String	O	In case super agent or Head qaurter is calling the API, sender can be used to filter branch transactions, otherwise transactions will be displayed for user wallet
receiver	String	O	Receiver wallet identifier (mobile number service name)
TransactionId	String	O	Search for specific transaction id
externalReference	String	O	External reference of transaction
username	String	O	Search for specific user transactions
dateFrom	String	M	dateTime format: dd/MM/yyyy HH:mm, dateFormat: dd/MM/yyyy
dateTo	String	O	dateTime format: dd/MM/yyyy HH:mm, dateFormat: dd/MM/yyyy
max	String	O	Page size , default 10, maximum 100
offset	String	O	Page offset, default 0
lang	String	O	AR for Arabic and EN for English : if not present, Arabic is selected
secureHash	String	M	See appendix 3

Request Example:

```
{ "msgId": "005", "lang": "AR", "secureHash": "23dfgj37dj38dsj8sd88d...." }
```

Response Example:

```
{ "desc": "Success", "errorCd": "00", "extraData": { "key": "txs", "value": { "reference": "2921218694761", "balanceAfter": "59863.67", "balanceBefore": "59867.17", "debitor": "SUPERSTAR", "processingStatusId": "1", "original Amount": "3.50", "totalAmount": "3.50", "senderCharge": "0.00", "commissionAmount": "0.00", "transDate": "19/08/2021 09:32:51", "isReversed": false, "senderCategory": "AGENT", "latinOperationName": "Buy Jawwal bundles initiated by agent", "localOperationName": "شراء حزم جوال عبر الوكيل", "mobileNumber": "0598511221", "senderDeviceName": "Galaxy A10", "senderDeviceId": "8510a842ee9c5e4a", "bundleCode": "D090_66771", "bundleName": "حزمة 200 دقيقة متجددة شهرية", "bundleTypeCode": "B090", "budnleType": "حزم دقائق متجددة" }, "ref": "1633592528138", "statusCode": "1", "totalCount": "1", "offset": "0" }
```



3 Calculate secure hash:

JawwalPay callbacks rely on HMAC to verify JawwalPay's identity and integrity of its data. In order to calculate the HMAC, please perform the following steps:

- 1- Sort the data dictionary by key Lexicographical order.
- 2- Concatenate the values (not the keys) in one string.
- 3- Calculate the hash of the concatenated string using SHA512 and your HMAC secret provided to you by jawwal pay team.
- 4- The resultant HMAC is Hex (base 16) lowercase.

For example,

suppose the below values are received from jawwal pay server excluding the secure_hash value :

`{"44393232930329", "00970598251590", "500", "EN" }`

The lexicographical order of the words concatenated is:

0097059825159044393232930329500EN

Suppose API KEY = 4Ffsk48gHgd49JvddAvNmZ

Hash the concatenated ordered values using the above secret =

de37b208e5e2e74e23fc00fc0984c0c2206fa10395c2b74aa6d5be36b379deb52cdb565a88363850 added3a697c8c0ec017bc900d47ee55852ce2f3afbf1cc1040

So, if the received secure_hash is the same as generated above, request will be accepted on jawwal pay side. Otherwise invalid secure hash error will be returned

4 Error Codes

Code	Name	Description
00	Success	Success
02	Internal System Error	Internal System Error
03	Unsupported Request	Unsupported Request
04	Sender Mobile Not Registered	Sender Mobile Not Registered
05	Receiver Mobile Not Registered	Receiver Mobile Not Registered
06	Agent Not Registered	Agent Not Registered
11	Invalid PIN	Invalid PIN
12	Insufficient Funds	Insufficient Funds
13	Agent Inactive	Agent Inactive
17	Transaction Not Found	Transaction Not Found
19	Request Timed Out	Request Timed Out
20	Exceeded Daily Count Limit	Exceeded Daily Count Limit
21	Exceeded Daily Amount Limit	Exceeded Daily Amount Limit
25	Sender Service Not Active	Sender Service Not Active
26	Sender Corporate Not Active	Sender Corporate Not Active
29	Sender Account Suspended	Sender Account Suspended
32	Receiver Account Suspended	Receiver Account Suspended
37	Failed to parse message	Failed to parse message
38	Service currently unavailable	Service currently unavailable
39	Sender Service Not Registered	Sender Service Not Registered
42	Invalid Device ID	Invalid Device ID
43	Invalid Activation Code	Invalid Activation Code
44	Invalid Password	Invalid Password
45	Device Not Found	Device Not Found
46	Message Is Duplicated	Message Is Duplicated
50	Device Not Active	Device Not Active
51	Device Is Stolen	Device Is Stolen
54	Device is already in use	Device is already in use
56	Invalid Mobile Number	Invalid Mobile Number
58	Transaction Amount Not Allowed	Transaction Amount Not Allowed
60	Exceeded Weekly Count Limit	Exceeded Weekly Count Limit
61	Exceeded Weekly Amount Limit	Exceeded Weekly Amount Limit
62	Exceeded Monthly Count Limit	Exceeded Monthly Count Limit
63	Exceeded Monthly Amount Limit	Exceeded Monthly Amount Limit
64	Exceeded Yearly Count Limit	Exceeded Yearly Count Limit
65	Exceeded Yearly Amount Limit	Exceeded Yearly Amount Limit
67	PaymentTypeNotSupported	Payment Type Not Supported
70	ServiceIntegrationFailure	Service Integration Failure
73	InvalidAmount	Invalid Amount
85	MessageNotFound	Message Not Found
86	InvalidOperation	Invalid Operation
88	ExpiredOTP	Expired OTP
89	InvalidOTP	Invalid OTP
90	DeviceBlocked	Device Blocked
91	SignInExpired	Sign In Expired
92	SenderAccountNotDefined	Sender Account Not Defined
93	ReceiverAccountNotDefined	Receiver Account Not Defined
94	SenderAccountInactive	Sender Account In Active
95	ReceiverAccountInactive	Receiver Account In Active

97	InvalidRequest	Invalid Request
99	CashCapExceeded	Cash Cap Exceeded
100	PasswordExpired	PasswordExpired
101	PinExpired	PinExpired
106	Wallet Cap Reached	Wallet Cap Reached
126	Exceed Number Of Attempts	Exceed Number Of Attempts
127	Max Number Of Requested Otp	Max Number Of Requested Otp
128	Max Number Of Retrying Otp	Max Number Of Retrying Otp
129	Max Number of Devices Reached	Max Number of Devices Reached
131	Activation Code Expired	Activation Code Expired
151	Exceeded Total Daily Count Limit	Exceeded Total Daily Count Limit
152	Exceeded Total Daily Amount Limit	Exceeded Total Daily Amount Limit
153	Exceeded Total Weekly Count Limit	Exceeded Total Weekly Count Limit
154	Exceeded Total Weekly Amount Limit	Exceeded Total Weekly Amount Limit
155	Exceeded Total Monthly Count Limit	Exceeded Total Monthly Count Limit
156	Exceeded Total Monthly Amount Limit	Exceeded Total Monthly Amount Limit
157	Exceeded Total Yearly Count Limit	Exceeded Total Yearly Count Limit
158	Exceeded Total Yearly Amount Limit	Exceeded Total Yearly Amount Limit
167	Invalid Account	Invalid Account
171	Exceeded Total Credit Limit	Exceeded Total Credit Limit
180	ServiceBlocked	ServiceBlocked
181	MAX_NUMBER_OF_ACTIVATION_CODE_REACHED	MAX_NUMBER_OF_ACTIVATION_CODE_REACHED
182	ACTIVATION_CODE_ALREADY_REQUESTED	ACTIVATION_CODE_ALREADY_REQUESTED

5 History of changes

Version	Editor	Date	Description
1.0	UBS	27/04/2022	Initial version
			-